Firmware and Mechanical 101 — polish, box, and what else

Lecture 3 • Fri 2 Oct 2026 • 14:20–16:20 • R311 IAMS

Your block's driver and phone panel, in sim mode (firmware that fakes its hardware, on the dev board) • one alarm rule • your box from the template • what else this can do

Where we are

- The five zones are merged; the board is **on order** since Mon 28 Sep, back ≈ 12 Oct, **fully assembled**. Here is the 3D view.
- Today everything runs in **sim mode on your dev board** (the bare ESP32-S3 board). The driver you write today runs unchanged on the real board in the wrap-up.
- **The demo checklist** (week of 26 Oct; format agreed in class): ① a waveform out, seen on the scope *and* on the instrument's own input • ② change it from the phone • ③ alarm → relay → notification • ④ **your block**, its panel, one thing it does, your number.

A block is one class with five methods

```
class OutputsBlock : public Block {
  const char* name() const override { return "b3"; }
  void begin() override;                          // pins, SPI — once
  void loop() override;                           // poll; fast; no delay()
  bool handle(JsonObjectConst cmd, JsonObject reply) override;   // one command
  void status(JsonObject out) override;           // your numbers, 20× per second
};
```

Rules: talks only to **its own hardware** · everything runs from `loop()` — no tasks, no locks, **no** `delay()` · `#ifdef SIM` fakes the hardware · real code is `TODO` until measured · pins only from `include/pins.h`.

Three data paths on one WebSocket

One **WebSocket** — a live two-way connection between phone and board — carries three kinds of traffic:

Command → reply (JSON — plain-text structured data)

`{"block":"b3","cmd":"sine","args":{"ch":1,"freq":1000,"amp":5}}`

→ `{"ok":true,"result":{…}}`

Status (JSON, 20 Hz)
every block's numbers → the strip chart, the alarm engine

Samples (binary, PROTOCOL § 6)

`stream {ch, rate_hz, chunk}` → rolling frames

`capture {ch, rate_hz, n, trig}` → one burst frame

→ the **Scope** tab: roll / single / auto, cursor, CSV

Rolling: tens of kS/s (WiFi). Burst: up to the ADC (500 kS/s).

`instrument.py` (the Python client on your PC) speaks all three. Your E2 `press` command

One round-trip, end to end

```
button.onclick → api.send('b3','sine',{ch:1,freq:1000,amp:5})
  → JSON text frame over ws://192.168.4.1/ws
    → main.cpp queues it → loop() → registry → OutputsBlock::handle()
      → writes the DAC (or, in SIM, remembers the setpoint)
    ← reply {"id":7,"ok":true,"result":{"ch":1,"mode":"sine"}}
  ← promise resolves → toast

every 50 ms: OutputsBlock::status(out) → {"ao1":…,"mode1":"sine"}
  → broadcast → panel.onStatus(st) → the number on your screen
```

You will explain this to the tutor **before your second flash** (writing the firmware onto the board).

One panel module per block

```
export default {
  id: 'b3', title: 'Outputs',
  render(el, api) {           // once, when the tab opens
    el.innerHTML = `…controls…`;
    el.querySelector('#sine').onclick = () =>
      api.send('b3', 'sine', { ch: 1, freq: 1000, amp: 5 }).catch(e => api.toast(e.message));
    el.querySelector('#alarm').onclick = () =>
      api.addAlarm({ block: 'b3', key: 'ao1', op: 'gt', threshold: 9.5, action: 'notify' });
  },
  onStatus(st) {              // 20× per second, your block's status object
    els.ao1.textContent = st.ao1.toFixed(3);
  },
};
```

Every status key you show must exist in your `status()`. Plain JavaScript, no build step, no CDN (nothing loaded from the internet) — the board is offline.

SIM mode and the alarm engine

SIM (`-DSIM=1`): your `#ifdef SIM` branch returns plausible, *moving* fake values — a slow sine, a counter, a drift towards the setpoint. Write it **first**; flash; see your tab; then the panel; then the real branch.

Alarms are data: `{block, key, op, threshold, action}` tested 20× per second on the same status; fire once on the rising edge; `notify` pops a message (a "toast") on every phone, `relay:<n>:on|off` acts on B4. Saved in `/alarms.json` .

Your one rule — B1 `ai1 > 9 → relay 1 off`

- B2 `v3v3 < 3.0 → notify` • B3 `ao1 > 9.5`
- B4 opto count jump • B5 TRIG edge.

E11 — driver + panel in sim (started now, finished at home)

1. Copy `blocks/template/` → `blocks/b<N>_<name>/` ; rename the class; `name()` → `"b<N>"`
2. **SIM branch first** — your status keys, your commands, moving fake values → flash `esp32s3-sim` → your tab shows them
3. `panels/template.js` → `panels/b<N>.js` — one control per command, one readout per key → `uploadfs` (sends the app's files to the board) → reload
4. **One alarm rule** from a button; trigger it in sim; watch the toast (and B4's relay flip)
5. Explain one round-trip to the tutor
6. **Pair up:** B3 ↔ B1 (a sine into an input) • B5 ↔ B4 (a TTL line into an opto)
7. Branch `b<N>-fw-<name>` (your own line of work) → pull request (PR); CI (GitHub's automatic build) builds both environments

Your block page lists your commands and keys: B1 `read_all set_range` • B2 `rails` • B3 `set_dc sine off` • B4 `relay`

Break

Then: boxes.

Boxes for electronics — six things to know

1. **FDM (filament 3D printing) tolerance:** holes **+0.3 mm**, slots +0.2 mm; a 3.0 mm screw wants a 3.3 mm hole.
2. **Walls 2 mm**; corners rounded; no unsupported overhang > 45°.
3. **Heat-set inserts** for M3 — print the boss, melt the insert in.
4. **Layer direction** is strength direction: print the lid flat; stand-offs vertical.
5. **The PCB front panel** makes the cutouts trivial: one rectangle, four holes. The SMAs and the OLED live on the panel PCB, not on the box.
6. **STEP (the 3D exchange file) from KiCad** (*File → Export → STEP*) — the template already contains the merged board.

Laser-cut acrylic is the alternative for flat panels. Not used for the box.

E12 — your box (started now, finished at home)

1. Open the instructor's **Onshape** (browser CAD) **template** → *Copy workspace* into your account
2. The board STEP is in it; the three parameters — **height, wall, foot style** — are in the *Variables* table
3. Change **one thing** that makes it yours: a vent pattern, an embossed name, different feet, a colour note. **One.**
4. *Export* → *STL* (the 3D-print file; binary, mm) → `docs/students/<name>/box.stl` → commit (save a snapshot), with a screenshot

The instructor prints all five on the Bambu the week after. Boards back ≈ 12 Oct go straight into them.

Build something your lab needs

The hardware was the baseline. **The software is yours.** The same shape — blocks, one socket, a phone panel, a Python client — becomes whatever your lab needs:

- **Bot alerts** — the alarm engine's `notify` → a PC script → LINE / Telegram webhook (a web address the bot listens on)
- **A data logger** — the Scope tab on a real signal; `instrument capture` straight into NumPy; CSV on a schedule
- **Scripted sweeps** — `instrument send b3 sine …` in a loop, `instrument stream b1.ai1 --csv`
- **A controller block** — close the loop between an input and an output; PID from day 1, on hardware
- **OTA** — update the firmware over WiFi, no cable ・ **a second node** — an S3-CAM watching a gauge
- **Videos, slides, reports with AI** — the lab's `show-your-work` skills (`education-video`

Homework 3 · then the wrap-up · then the demo

Homework 3 — hand-in dates agreed in class
E11 driver + panel PR — all commands, one value plotted, the alarm rule, CI green; ring review comment-only (V6 first)
E12 `box.stl` in your folder
V6 + V7, chapter 3 reading

Wrap-up (week of 19 Oct): flash the real board without SIM · your panel moves real hardware · **one measured number** with its method into `SPEC.md` · 60-second video.

Demo (week of 26 Oct): the four-item checklist, video as the safety net; optional item 5 — what you built beyond the baseline (format agreed in class).

Build it. Ship it. Log it.

You did it — built, published where others can open it, logged — on a teaching page, on a dev board, on a class board, in firmware, in a box.
The next instrument is yours.